

# ConsultIQ API Service Contract

## OpenAPI 3.0.3 Specification

### ConsultIQ

## Introduction

This document defines the ConsultIQ API Service Contract as a formal OpenAPI 3.0.3 specification. It establishes the boundary interface between the frontend and backend, specifying every endpoint, request/response schema, authentication mechanism, and error-handling behaviour, so that client and server developers can build independently without breaking integration.

## OpenAPI Specification

```
openapi: 3.0.3
info:
  title: ConsultIQ API
  version: 1.0.0
  description: >
    Service contract defining endpoints for the ConsultIQ platform.
    Covers:
    (R1)Authentication,
    (R2) Consultant Profile Management,
    (R3) Project Specification Management,
    (R4) Scoring Configuration Management,
    (R5) Match Run Execution,
    (R6) Notification Management,
    (R7) Administration ,
    supporting Placement,
    CV, Location, Health, and Analytics endpoints.
servers:
  - url: https://api.consultiq.co.za/v1

security:
  - bearerAuth: []

components:
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT

  schemas:
    Error:
      type: object
```

```

properties:
  error:
    type: string
    example: "ERROR_CODE"
  message:
    type: string
    example: "Human-readable message"

ScoringFactor:
  type: object
  properties:
    factorName: { type: string, example: "SKILL_ALIGNMENT" }
    weight: { type: number, example: 40 }
    active: { type: boolean }
    hardExclusionEnabled: { type: boolean }

ScoringOverride:
  type: object
  properties:
    factorName: { type: string }
    overrideWeight: { type: number }
    active: { type: boolean }
    hardExclusionEnabled: { type: boolean }

Notification:
  type: object
  properties:
    id: { type: string, format: uuid }
    userId: { type: string, format: uuid }
    title: { type: string }
    body: { type: string }
    link: { type: string }
    isRead: { type: boolean }
    isArchived: { type: boolean }
    archivedAt: { type: string, format: date-time }
    createdAt: { type: string, format: date-time }

MatchResult:
  type: object
  properties:
    consultantId: { type: string, format: uuid }
    consultantName: { type: string }
    consultantEmail: { type: string }
    finalScore: { type: number, example: 87.5 }
    rank: { type: integer }
    factorBreakdown:
      type: array
      items:
        type: object
        properties:
          factorName: { type: string }
          weight: { type: number }
          rawScore: { type: number }
          weightedScore: { type: number }
    isPlaced: { type: boolean }

```

```

responses:
  Unauthorized:
    description: Authentication token missing or expired.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
        example:
          error: UNAUTHORISED
          message: Authentication is required. Please log in.
  Forbidden:
    description: User does not have the required role permission for this action.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
        example:
          error: FORBIDDEN
          message: You do not have permission to perform this action.
  NotFound:
    description: The requested resource could not be found.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'
        example:
          error: NOT_FOUND
          message: The requested resource could not be found.
  ValidationError:
    description: The request body failed validation.
    content:
      application/json:
        schema:
          $ref: '#/components/schemas/Error'

paths:
  # ----- R1: Authentication -----
  /auth/register:
    post:
      tags: [Authentication]
      summary: Register a new pending user account
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [fullName, email, role]
              properties:
                fullName: { type: string, example: "Jane Smith" }
                email: { type: string, format: email, example: "jane.smith@consultiq.com" }
                ↵ }

```

```

        role: { type: string, example: "CONSULTANT" }
responses:
  '201':
    description: User created, pending verification.
    content:
      application/json:
        schema:
          type: object
          properties:
            userId: { type: string, format: uuid }
            email: { type: string }
            role: { type: string }
            status: { type: string, example: "PENDING_VERIFICATION" }
            message: { type: string }
  '409':
    description: A user with this email already exists.
    content:
      application/json:
        schema: { $ref: '#/components/schemas/Error' }
  '400': { $ref: '#/components/responses/ValidationError' }

/auth/complete-registration:
  post:
    tags: [Authentication]
    summary: Complete onboarding for a registered user
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [token, username, password, acceptedPopia]
            properties:
              token: { type: string }
              username: { type: string }
              password: { type: string, format: password }
              acceptedPopia: { type: boolean }
    responses:
      '200':
        description: Account activated successfully.
        content:
          application/json:
            schema:
              type: object
              properties:
                userId: { type: string, format: uuid }
                username: { type: string }
                status: { type: string, example: "ACTIVE" }
                message: { type: string }
      '400':
        description: Invalid token or POPIA consent not accepted.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

```

```

/popia:
  get:
    tags: [Authentication]
    summary: Retrieve POPIA consent content
    security: []
    responses:
      '200':
        description: POPIA content retrieved successfully.
        content:
          application/json:
            schema:
              type: object
              properties:
                title: { type: string }
                content: { type: string }

/auth/login:
  post:
    tags: [Authentication]
    summary: Authenticate a user
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email, password]
            properties:
              email: { type: string, format: email }
              password: { type: string, format: password }
    responses:
      '200':
        description: Login successful.
        content:
          application/json:
            schema:
              type: object
              properties:
                token: { type: string }
                expiresIn: { type: integer, example: 28800 }
                user:
                  type: object
                  properties:
                    id: { type: string, format: uuid }
                    fullName: { type: string }
                    role: { type: string }
      '401':
        description: Invalid credentials.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

/auth/logout:

```

```

post:
  tags: [Authentication]
  summary: Invalidate the current session token
  responses:
    '200':
      description: Logout successful.
    '401': { $ref: '#/components/responses/Unauthorized' }

/auth/activate:
post:
  tags: [Authentication]
  summary: Activate a pending user account via activation token
  security: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          required: [token]
          properties:
            token: { type: string }
  responses:
    '200':
      description: Account activated successfully.
    '400':
      description: Invalid or expired activation token.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }

/auth/resend-verification:
post:
  tags: [Authentication]
  summary: Resend the account verification email
  security: []
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          required: [email]
          properties:
            email: { type: string, format: email }
  responses:
    '200':
      description: Verification email resent successfully.
    '404': { $ref: '#/components/responses/NotFound' }

/auth/accept-terms:
post:
  tags: [Authentication]
  summary: Record acceptance of updated terms and conditions
  requestBody:

```

```

    required: true
  content:
    application/json:
      schema:
        type: object
        required: [accepted]
        properties:
          accepted: { type: boolean }
  responses:
    '200':
      description: Terms accepted successfully.
    '401': { $ref: '#/components/responses/Unauthorized' }

/auth/forgot-password:
  post:
    tags: [Authentication]
    summary: Initiate the password reset process
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [email]
            properties:
              email: { type: string, format: email }
    responses:
      '200':
        description: Reset link sent if the account exists.

/auth/reset-password:
  post:
    tags: [Authentication]
    summary: Reset a user's password using a reset token
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [token, newPassword]
            properties:
              token: { type: string }
              newPassword: { type: string, format: password }
    responses:
      '200':
        description: Password reset successfully.
      '400':
        description: Invalid or expired reset token.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

```

```

/auth/refresh:
  post:
    tags: [Authentication]
    summary: Issue a new JWT using a refresh token
    security: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            required: [refreshToken]
            properties:
              refreshToken: { type: string }
    responses:
      '200':
        description: New token issued.
        content:
          application/json:
            schema:
              type: object
              properties:
                token: { type: string }
                expiresIn: { type: integer, example: 28800 }
      '401':
        description: Invalid or expired refresh token.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

/auth/me:
  get:
    tags: [Authentication]
    summary: Get the current authenticated user's profile
    responses:
      '200':
        description: Current user profile.
        content:
          application/json:
            schema:
              type: object
              properties:
                id: { type: string, format: uuid }
                fullName: { type: string }
                email: { type: string }
                role: { type: string }
                status: { type: string }
      '401': { $ref: '#/components/responses/Unauthorized' }

# ----- R2: Consultant Profile Management -----
/consultants:
  get:
    tags: [Consultants]
    summary: List all consultants (paginated, searchable, filterable)

```



description: Access Control -- Consultant Manager only. Search is case-insensitive;  
↪ filters use AND logic. Cost-to-company excluded from list view.

parameters:

- in: query  
name: page  
schema: { type: integer }
- in: query  
name: search  
schema: { type: string }

responses:

'200':  
description: Paginated list of consultants.  
content:  
application/json:  
schema:  
type: object  
properties:  
page: { type: integer }  
total: { type: integer }  
consultants:  
type: array  
items:  
type: object  
properties:  
id: { type: string, format: uuid }  
fullName: { type: string }  
city: { type: string }  
availabilityStatus: { type: string }  
primarySkills:  
type: array  
items: { type: string }  
'401': { \$ref: '#/components/responses/Unauthorized' }  
'403': { \$ref: '#/components/responses/Forbidden' }

/consultants/create:

post:

tags: [Consultants]  
summary: Create a new consultant profile  
description: Access Control -- Consultant Manager only. Skills and certifications  
↪ saved atomically; duplicate skills not allowed.

requestBody:

required: true  
content:  
application/json:  
schema:  
type: object  
properties:  
fullName: { type: string }  
email: { type: string, format: email }  
city: { type: string }  
province: { type: string }  
costToCompanyRate: { type: number }  
availabilityStatus: { type: string }  
skills:  
type: array

```

        items:
          type: object
          properties:
            skillName: { type: string }
            yearsOfExperience: { type: number }
            confidenceLevel: { type: integer }
        certifications:
          type: array
          items:
            type: object
            properties:
              certificationName: { type: string }
              issuingBody: { type: string }
responses:
  '201':
    description: Consultant profile created successfully.
    content:
      application/json:
        schema:
          type: object
          properties:
            message: { type: string }
            consultantId: { type: string, format: uuid }
  '400': { $ref: '#/components/responses/ValidationError' }
  '401': { $ref: '#/components/responses/Unauthorized' }
  '403': { $ref: '#/components/responses/Forbidden' }

/consultants/profile:
  post:
    tags: [Consultants]
    summary: Submit own consultant profile for review
    description: Access Control -- Consultant only.
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              fullName: { type: string }
              city: { type: string }
              province: { type: string }
              skills:
                type: array
                items:
                  type: object
                  properties:
                    skillName: { type: string }
                    yearsOfExperience: { type: number }
                    confidenceLevel: { type: integer }
    responses:
      '201':
        description: Profile submitted for review.
        content:
          application/json:

```

```

        schema:
          type: object
          properties:
            message: { type: string }
            consultantId: { type: string, format: uuid }
            status: { type: string, example: "PENDING_APPROVAL" }

/consultants/pending-profiles:
  get:
    tags: [Consultants]
    summary: List consultant profiles awaiting approval
    description: Access Control -- Consultant Manager only.
    responses:
      '200':
        description: Paginated list of pending consultant profiles.
        content:
          application/json:
            schema:
              type: object
              properties:
                page: { type: integer }
                total: { type: integer }
                consultants:
                  type: array
                  items:
                    type: object
                    properties:
                      id: { type: string, format: uuid }
                      fullName: { type: string }
                      status: { type: string }
      '403': { $ref: '#/components/responses/Forbidden' }

/consultants/competency-preview:
  post:
    tags: [Consultants]
    summary: Preview a derived competency level (no data persisted)
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              yearsOfExperience: { type: number }
              confidenceLevel: { type: integer }
    responses:
      '200':
        description: Derived competency level.
        content:
          application/json:
            schema:
              type: object
              properties:
                competencyLevel: { type: string }
      '400': { $ref: '#/components/responses/ValidationError' }

```

```

/consultants/assigned/project:
  get:
    tags: [Consultants]
    summary: Get the authenticated consultant's current project assignment
    description: Access Control -- Consultant only.
    responses:
      '200':
        description: Active project assignment.
        content:
          application/json:
            schema:
              type: object
              properties:
                projectId: { type: string, format: uuid }
                projectName: { type: string }
                allocationPercentage: { type: number }
      '404': { $ref: '#/components/responses/NotFound' }

/consultants/assigned/projects/{projectId}:
  get:
    tags: [Consultants]
    summary: List consultants assigned to a project
    description: Access Control -- Project Manager only.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Assigned consultants for the project.
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  consultantId: { type: string, format: uuid }
                  fullName: { type: string }
                  allocationPercentage: { type: number }
      '404': { $ref: '#/components/responses/NotFound' }

/consultants/project/{projectId}:
  get:
    tags: [Consultants]
    summary: Get consultants associated with a project (matched and placed)
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':

```

```

    description: Consultants linked to the project.
    content:
      application/json:
        schema:
          type: array
          items:
            type: object
            properties:
              consultantId: { type: string, format: uuid }
              fullName: { type: string }
              isPlaced: { type: boolean }

/consultants/project/{projectId}/unassign/{consultantId}:
  patch:
    tags: [Consultants]
    summary: Unassign a consultant from a project
    description: Access Control -- Project Manager only.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
      - in: path
        name: consultantId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Consultant unassigned successfully.
      '404': { $ref: '#/components/responses/NotFound' }

/consultants/user/{userId}:
  get:
    tags: [Consultants]
    summary: Get a consultant profile by associated user ID
    parameters:
      - in: path
        name: userId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Consultant profile.
        content:
          application/json:
            schema:
              type: object
              properties:
                id: { type: string, format: uuid }
                userId: { type: string, format: uuid }
                fullName: { type: string }
      '404': { $ref: '#/components/responses/NotFound' }

/consultants/{consultantId}:
  get:

```

```

tags: [Consultants]
summary: Get full details of a consultant profile
description: Access Control -- Consultant Manager and/or Admin.
parameters:
  - in: path
    name: consultantId
    required: true
    schema: { type: string, format: uuid }
responses:
  '200':
    description: Full consultant profile.
    content:
      application/json:
        schema:
          type: object
          properties:
            id: { type: string, format: uuid }
            fullName: { type: string }
            email: { type: string }
            city: { type: string }
            province: { type: string }
            costToCompanyRate: { type: number }
            availabilityStatus: { type: string }
            skills:
              type: array
              items:
                type: object
                properties:
                  skillName: { type: string }
                  yearsOfExperience: { type: number }
                  confidenceLevel: { type: integer }
                  competencyLevel: { type: string }
            certifications:
              type: array
              items: { type: object }
  '401': { $ref: '#/components/responses/Unauthorized' }
  '403': { $ref: '#/components/responses/Forbidden' }
  '404': { $ref: '#/components/responses/NotFound' }
put:
  tags: [Consultants]
  summary: Fully update a consultant profile
  description: Access Control -- Consultant Managers and Consultants (consultant-made
    ↪ changes need approval). Skill set replaced atomically; competency levels
    ↪ recalculated.
  parameters:
    - in: path
      name: consultantId
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object

```

```

    properties:
      fullName: { type: string }
      email: { type: string }
      city: { type: string }
      province: { type: string }
      costToCompanyRate: { type: number }
      availabilityStatus: { type: string }
      skills:
        type: array
        items:
          type: object
          properties:
            skillName: { type: string }
            yearsOfExperience: { type: number }
            confidenceLevel: { type: integer }
      certifications:
        type: array
        items: { type: object }
  responses:
    '200':
      description: Consultant updated successfully.
      content:
        application/json:
          schema:
            type: object
            properties:
              message: { type: string }
              consultantId: { type: string, format: uuid }
    '400': { $ref: '#/components/responses/ValidationError' }
  patch:
    tags: [Consultants]
    summary: Partially update a consultant profile
    description: Access Control -- Consultant Managers and Consultants (consultant-made
    ↪ changes need approval).
    parameters:
      - in: path
        name: consultantId
        required: true
        schema: { type: string, format: uuid }
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              availabilityStatus: { type: string }
    responses:
      '200':
        description: Consultant updated successfully.
        content:
          application/json:
            schema:
              type: object
              properties:

```

```

        message: { type: string }
        consultantId: { type: string, format: uuid }
delete:
  tags: [Consultants]
  summary: Remove a consultant profile
  description: Access Control -- Consultant Manager and Admin.
  parameters:
    - in: path
      name: consultantId
      required: true
      schema: { type: string, format: uuid }
  responses:
    '200':
      description: Consultant removed successfully.
    '404': { $ref: '#/components/responses/NotFound' }

/consultants/{id}/picture:
  post:
    tags: [Consultants]
    summary: Upload or replace a consultant's profile picture
    description: Access Control -- Consultant Managers and Consultants.
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            properties:
              file:
                type: string
                format: binary
    responses:
      '200':
        description: Profile picture uploaded successfully.
        content:
          application/json:
            schema:
              type: object
              properties:
                message: { type: string }
                pictureUrl: { type: string, format: uri }
      '400': { $ref: '#/components/responses/ValidationError' }

# ----- R3: Project Specification Management -----
/projects:
  get:
    tags: [Projects]
    summary: List all project specifications (paginated)
    description: Access Control -- Project Manager only.
    responses:

```



```

'200':
  description: List of project specifications.
  content:
    application/json:
      schema:
        type: object
        properties:
          page: { type: integer }
          total: { type: integer }
          projects:
            type: array
            items:
              type: object
              properties:
                id: { type: string, format: uuid }
                projectName: { type: string }
                clientName: { type: string }
                city: { type: string }
                province: { type: string }
                status: { type: string }
                startDate: { type: string, format: date }
                requiredSkillCount: { type: integer }

post:
  tags: [Projects]
  summary: Create a new project specification
  description: Access Control -- Project Manager only. Initial status is always
  ↪ "Open".
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            projectName: { type: string }
            clientName: { type: string }
            city: { type: string }
            province: { type: string }
            startDate: { type: string, format: date }
            endDate: { type: string, format: date }
            teamSize: { type: integer }
            requiredAllocationPercentage: { type: number }
            clientBillingBudget: { type: number }
            skillRequirements:
              type: array
              items:
                type: object
                properties:
                  skillName: { type: string }
                  minimumCompetencyLevel: { type: string }
                  mandatory: { type: boolean }

  responses:
    '200':
      description: Project specification created with status Open.
      content:

```

```

    application/json:
      schema:
        type: object
        properties:
          message: { type: string }
          projectId: { type: string, format: uuid }
          status: { type: string, example: "Open" }
        '400': { $ref: '#/components/responses/ValidationError' }
        '403': { $ref: '#/components/responses/Forbidden' }

/projects/{projectId}:
  get:
    tags: [Projects]
    summary: Get full details of a project specification
    description: Access Control -- Project Manager only.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Full project specification.
        content:
          application/json:
            schema:
              type: object
              properties:
                id: { type: string, format: uuid }
                projectName: { type: string }
                clientName: { type: string }
                city: { type: string }
                province: { type: string }
                startDate: { type: string, format: date }
                endDate: { type: string, format: date }
                teamSize: { type: integer }
                requiredAllocationPercentage: { type: number }
                clientBillingBudget: { type: number }
                status: { type: string }
                skillRequirements:
                  type: array
                  items:
                    type: object
                    properties:
                      skillName: { type: string }
                      minimumCompetencyLevel: { type: string }
                      mandatory: { type: boolean }
              '404': { $ref: '#/components/responses/NotFound' }
      '404': { $ref: '#/components/responses/NotFound' }
  put:
    tags: [Projects]
    summary: Fully update a project specification
    description: Access Control -- Project Manager only.
    parameters:
      - in: path
        name: projectId

```

```

    required: true
    schema: { type: string, format: uuid }
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        properties:
          projectName: { type: string }
          clientName: { type: string }
          city: { type: string }
          province: { type: string }
          startDate: { type: string, format: date }
          endDate: { type: string, format: date }
          teamSize: { type: integer }
          requiredAllocationPercentage: { type: number }
          clientBillingBudget: { type: number }
          skillRequirements:
            type: array
            items:
              type: object
              properties:
                skillName: { type: string }
                minimumCompetencyLevel: { type: string }
                mandatory: { type: boolean }
responses:
  '200':
    description: Project updated successfully.
    content:
      application/json:
        schema:
          type: object
          properties:
            message: { type: string }
            projectId: { type: string, format: uuid }
  '400': { $ref: '#/components/responses/ValidationError' }
  '403': { $ref: '#/components/responses/Forbidden' }
patch:
  tags: [Projects]
  summary: Partially update a project specification
  description: Access Control -- Project Manager only.
  parameters:
    - in: path
      name: projectId
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            teamSize: { type: integer }

```

```

responses:
  '200':
    description: Project updated successfully.
    content:
      application/json:
        schema:
          type: object
          properties:
            message: { type: string }
            projectId: { type: string, format: uuid }

/projects/{projectId}/status:
  patch:
    tags: [Projects]
    summary: Update the status of a project specification
    description: Access Control -- Project Manager only.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              status: { type: string, example: "In Progress" }
    responses:
      '200':
        description: Project status updated successfully.
        content:
          application/json:
            schema:
              type: object
              properties:
                message: { type: string }
                projectId: { type: string, format: uuid }
                status: { type: string }
      '400':
        description: Invalid project status.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      '403': { $ref: '#/components/responses/Forbidden' }

# ----- R4: Scoring Configuration Management -----
/scoring-config:
  get:
    tags: [Scoring Configuration]
    summary: Get the firm-wide scoring configuration
    description: >
      Access Control -- Admin only. If no configuration exists, the system seeds
      and returns default factors (Skill Alignment, Competency Level, Availability,

```

Location, Cost to Company). Note: logs show this endpoint mounted at /config/scoring -- confirm the intended base path with the team.

responses:

```

'200':
  description: Current scoring configuration.
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/ScoringFactor'
'401': { $ref: '#/components/responses/Unauthorized' }

```

put:

```

tags: [Scoring Configuration]
summary: Update the firm-wide scoring configuration
description: Access Control -- Admin only. Active factor weights must sum to
↪ exactly 100. Every update is written to an audit log.
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        properties:
          scoringFactors:
            type: array
            items:
              $ref: '#/components/schemas/ScoringFactor'

```

responses:

```

'200':
  description: Updated scoring factors.
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/ScoringFactor'
'400':
  description: Active factor weights do not sum to 100, or no active factors
↪ supplied.
  content:
    application/json:
      schema: { $ref: '#/components/schemas/Error' }

```

/projects/{projectId}/scoring-override:

get:

```

tags: [Scoring Configuration]
summary: Get project-specific scoring factor overrides
description: Access Control -- Assigned Project Manager only.
parameters:
  - in: path
    name: projectId
    required: true
    schema: { type: string, format: uuid }

```

responses:

```

'200':
  description: Project scoring overrides.
  content:
    application/json:
      schema:
        type: array
        items:
          $ref: '#/components/schemas/ScoringOverride'
'403':
  description: Only the assigned Project Manager can manage scoring overrides for
    ↪ this project.
  content:
    application/json:
      schema: { $ref: '#/components/schemas/Error' }
put:
  tags: [Scoring Configuration]
  summary: Create or update project-specific scoring factor overrides
  description: Access Control -- Assigned Project Manager only. Active override
    ↪ weights must sum to exactly 100.
  parameters:
    - in: path
      name: projectId
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            factors:
              type: array
              items:
                $ref: '#/components/schemas/ScoringOverride'
  responses:
    '200':
      description: Updated project scoring overrides.
      content:
        application/json:
          schema:
            type: array
            items:
              $ref: '#/components/schemas/ScoringOverride'
    '400':
      description: Active override weights do not sum to exactly 100.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }
    '403':
      description: Only the assigned Project Manager can manage scoring overrides for
        ↪ this project.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }

```

```

delete:
  tags: [Scoring Configuration]
  summary: Remove project-specific scoring factor overrides
  description: Access Control -- Assigned Project Manager only. Deletion must be
    ↪ explicitly confirmed; reverts project to firm-wide configuration.
  parameters:
    - in: path
      name: projectId
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true
    content:
      application/json:
        schema:
          type: object
          properties:
            confirm: { type: boolean }
  responses:
    '200':
      description: Project scoring override removed successfully.
    '400':
      description: Deletion must be explicitly confirmed.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }
    '403':
      description: Only the assigned Project Manager can manage scoring overrides for
        ↪ this project.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }

# ----- R5: Match Run Execution -----
/projects/{id}/match-run:
  post:
    tags: [Match Run]
    summary: Execute a new match run for a project
    description: >
      Access Control -- Authenticated user (assigned Project Manager). Active
      consultants are scored using resolved scoring weights (project override,
      falling back to firm-wide configuration). Consultants with an overlapping
      active placement are flagged isPlaced. The scoring weight configuration
      used is snapshotted on the match run record.
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    responses:
      '201':
        description: Match run executed and results persisted.
        content:
          application/json:
            schema:

```

```

        type: object
        properties:
            runId: { type: string, format: uuid }
            results:
                type: array
                items:
                    $ref: '#/components/schemas/MatchResult'
    '400':
        description: Project has no required skills, invalid project status, or no
        ↪ eligible consultants.
        content:
            application/json:
                schema: { $ref: '#/components/schemas/Error' }
    '404':
        description: Project does not exist.
        content:
            application/json:
                schema: { $ref: '#/components/schemas/Error' }

/projects/{id}/match-run/{runId}:
get:
    tags: [Match Run]
    summary: Get the ranked, scored results of a match run
    parameters:
        - in: path
          name: id
          required: true
          schema: { type: string, format: uuid }
        - in: path
          name: runId
          required: true
          schema: { type: string, format: uuid }
    responses:
        '200':
            description: Ranked match run results.
            content:
                application/json:
                    schema:
                        type: array
                        items:
                            $ref: '#/components/schemas/MatchResult'
        '404': { $ref: '#/components/responses/NotFound' }

/projects/{id}/match-run/{runId}/status:
get:
    tags: [Match Run]
    summary: Get the processing status of a match run
    description: Distinct from scored results -- reflects Pending, Running, Completed,
    ↪ or Failed state.
    parameters:
        - in: path
          name: id
          required: true
          schema: { type: string, format: uuid }
        - in: path

```



```

        name: runId
        required: true
        schema: { type: string, format: uuid }
responses:
  '200':
    description: Match run processing status.
    content:
      application/json:
        schema:
          type: object
          properties:
            runId: { type: string, format: uuid }
            status: { type: string, example: "COMPLETED" }
  '404': { $ref: '#/components/responses/NotFound' }

/projects/{id}/match-run/{runId}/stats:
get:
  tags: [Match Run]
  summary: Get summary statistics for a match run
  parameters:
    - in: path
      name: id
      required: true
      schema: { type: string, format: uuid }
    - in: path
      name: runId
      required: true
      schema: { type: string, format: uuid }
  responses:
    '200':
      description: Match run statistics.
      content:
        application/json:
          schema:
            type: object
            properties:
              totalEvaluated: { type: integer }
              totalExcluded: { type: integer }
              totalMatched: { type: integer }
              totalPlaced: { type: integer }
    '404': { $ref: '#/components/responses/NotFound' }

# ----- Placement Management -----
/projects/{id}/placements:
post:
  tags: [Placements]
  summary: Create a placement, assigning a consultant to a project
  description: Access Control -- Project Manager only. Typically follows a match run.
  parameters:
    - in: path
      name: id
      required: true
      schema: { type: string, format: uuid }
  requestBody:
    required: true

```

```

    content:
      application/json:
        schema:
          type: object
          properties:
            consultantId: { type: string, format: uuid }
            allocationPercentage: { type: number }
            startDate: { type: string, format: date }
  responses:
    '201':
      description: Placement created successfully.
      content:
        application/json:
          schema:
            type: object
            properties:
              message: { type: string }
              placementId: { type: string, format: uuid }
    '400':
      description: Consultant already has an overlapping active placement.
      content:
        application/json:
          schema: { $ref: '#/components/schemas/Error' }

# ----- R6: Notification Management -----
/notifications:
  get:
    tags: [Notifications]
    summary: Get the authenticated user's recent notifications
    description: Non-archived, newest to oldest, capped at 50 records.
    responses:
      '200':
        description: List of notifications.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Notification'
      '401': { $ref: '#/components/responses/Unauthorized' }

/notifications/archived:
  get:
    tags: [Notifications]
    summary: Get the authenticated user's archived notifications
    description: Newest to oldest, capped at 50 records.
    responses:
      '200':
        description: List of archived notifications.
        content:
          application/json:
            schema:
              type: array
              items:
                $ref: '#/components/schemas/Notification'

```

```

    '401': { $ref: '#/components/responses/Unauthorized' }

/notifications/read-all:
  patch:
    tags: [Notifications]
    summary: Mark all notifications as read
    responses:
      '200':
        description: All notifications marked as read.
        content:
          application/json:
            schema:
              type: object
              properties:
                message: { type: string }
                updatedCount: { type: integer }
      '401': { $ref: '#/components/responses/Unauthorized' }

/notifications/{id}/read:
  patch:
    tags: [Notifications]
    summary: Mark a single notification as read
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Notification marked as read.
        content:
          application/json:
            schema:
              type: object
              properties:
                id: { type: string, format: uuid }
                isRead: { type: boolean }
      '404': { $ref: '#/components/responses/NotFound' }

/notifications/{id}/archive:
  patch:
    tags: [Notifications]
    summary: Archive a single notification
    parameters:
      - in: path
        name: id
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Notification archived.
        content:
          application/json:
            schema:
              type: object

```

```

        properties:
            id: { type: string, format: uuid }
            isArchived: { type: boolean }
            archivedAt: { type: string, format: date-time }
        '404': { $ref: '#/components/responses/NotFound' }

# ----- CV Management -----
/cv/upload/{userId}:
  post:
    tags: [CV]
    summary: Upload a CV for a user and queue it for parsing
    parameters:
      - in: path
        name: userId
        required: true
        schema: { type: string, format: uuid }
    requestBody:
      required: true
      content:
        multipart/form-data:
          schema:
            type: object
            properties:
              file:
                type: string
                format: binary
    responses:
      '201':
        description: CV uploaded and queued for parsing.
        content:
          application/json:
            schema:
              type: object
              properties:
                cvFileId: { type: string, format: uuid }
                message: { type: string }
      '400':
        description: Unsupported file type.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }

/cv/{cvFileId}/url:
  get:
    tags: [CV]
    summary: Get a signed download URL for a CV file
    parameters:
      - in: path
        name: cvFileId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Signed, time-limited download URL.
        content:

```

```

        application/json:
          schema:
            type: object
            properties:
              url: { type: string, format: uri }
              expiresIn: { type: integer, example: 900 }
            '404': { $ref: '#/components/responses/NotFound' }

/cv/{cvFileId}:
  get:
    tags: [CV]
    summary: Get metadata for an uploaded CV file
    parameters:
      - in: path
        name: cvFileId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: CV file metadata.
        content:
          application/json:
            schema:
              type: object
              properties:
                cvFileId: { type: string, format: uuid }
                fileName: { type: string }
                parsingStatus: { type: string }
                uploadedAt: { type: string, format: date-time }
            '404': { $ref: '#/components/responses/NotFound' }
      '404': { $ref: '#/components/responses/NotFound' }
  delete:
    tags: [CV]
    summary: Delete a CV file and its metadata
    parameters:
      - in: path
        name: cvFileId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: CV deleted successfully.
      '404': { $ref: '#/components/responses/NotFound' }

# ----- Location -----
/location/search:
  get:
    tags: [Location]
    summary: Search cities/provinces for autocomplete
    parameters:
      - in: query
        name: q
        schema: { type: string }
    responses:
      '200':
        description: Matching locations.

```

```

    content:
      application/json:
        schema:
          type: array
          items:
            type: object
            properties:
              city: { type: string }
              province: { type: string }

# ----- Health -----
/health:
  get:
    tags: [Health]
    summary: Get operational status of the API and its dependencies
    security: []
    responses:
      '200':
        description: Health check result.
        content:
          application/json:
            schema:
              type: object
              properties:
                status: { type: string, example: "ok" }
                info:
                  type: object
                  properties:
                    database:
                      type: object
                      properties:
                        status: { type: string }
                    redis:
                      type: object
                      properties:
                        status: { type: string }

# ----- R7: Administration -----
/admin/users:
  get:
    tags: [Administration]
    summary: List non-admin users (paginated)
    description: Access Control -- Admin only. Excludes users still pending
    ↪ verification.
    responses:
      '200':
        description: Paginated user list with aggregate counts.
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:

```

```

        type: object
        properties:
            id: { type: string, format: uuid }
            fullName: { type: string }
            email: { type: string }
            role: { type: string }
            status: { type: string }
            createdAt: { type: string, format: date-time }
    meta:
        type: object
        properties:
            totalRecords: { type: integer }
            currentPage: { type: integer }
            totalPages: { type: integer }
            activeUsers: { type: integer }
            suspendedUsers: { type: integer }
    '403': { $ref: '#/components/responses/Forbidden' }

/admin/users/{userId}/activate:
    patch:
        tags: [Administration]
        summary: Activate a target user account
        description: Access Control -- Admin only.
        parameters:
            - in: path
              name: userId
              required: true
              schema: { type: string, format: uuid }
        responses:
            '200':
                description: User activated successfully.
            '404': { $ref: '#/components/responses/NotFound' }

/admin/users/{userId}/suspend:
    patch:
        tags: [Administration]
        summary: Suspend a target user account
        description: Access Control -- Admin only.
        parameters:
            - in: path
              name: userId
              required: true
              schema: { type: string, format: uuid }
        responses:
            '200':
                description: User suspended successfully.
            '404': { $ref: '#/components/responses/NotFound' }

/admin/users/{userId}/role:
    put:
        tags: [Administration]
        summary: Change the role of a target user
        description: >
            Access Control -- Super Admin and Admin only. An admin cannot change their
            own role, cannot assign SUPER_ADMIN, and cannot change an existing

```

```

    SUPER_ADMIN's role. Every change is written to an admin audit log.
parameters:
  - in: path
    name: userId
    required: true
    schema: { type: string, format: uuid }
requestBody:
  required: true
  content:
    application/json:
      schema:
        type: object
        properties:
          role: { type: string, example: "CONSULTANT_MANAGER" }
responses:
  '200':
    description: Role changed successfully.
    content:
      application/json:
        schema:
          type: object
          properties:
            message: { type: string }
  '400':
    description: Attempted to change own role or assign SUPER_ADMIN.
    content:
      application/json:
        schema: { $ref: '#/components/schemas/Error' }
  '404':
    description: Target user not found.
    content:
      application/json:
        schema: { $ref: '#/components/schemas/Error' }

/admin/projects:
  get:
    tags: [Administration]
    summary: List non-archived project specifications for oversight
    description: Access Control -- Admin only.
    responses:
      '200':
        description: Paginated project list with record counts.
        content:
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    type: object
                    properties:
                      id: { type: string, format: uuid }
                      projectName: { type: string }
                      status: { type: string }

```



```

        clientName: { type: string }
        createdAt: { type: string, format: date-time }
        budget: { type: number }
    meta:
        type: object
        properties:
            totalRecords: { type: integer }
            absoluteTotalRecords: { type: integer }
            currentPage: { type: integer }
            totalPages: { type: integer }
    '403': { $ref: '#/components/responses/Forbidden' }

/admin/projects/{projectId}/archive:
  patch:
    tags: [Administration]
    summary: Archive a project specification
    description: Access Control -- Admin only. Recorded in the project audit log.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Project archived successfully.
      '401':
        description: Admin identity could not be resolved from the request.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      '404': { $ref: '#/components/responses/NotFound' }

/admin/projects/{projectId}/unarchive:
  patch:
    tags: [Administration]
    summary: Restore a previously archived project to Open status
    description: Access Control -- Admin only. Recorded in the project audit log.
    parameters:
      - in: path
        name: projectId
        required: true
        schema: { type: string, format: uuid }
    responses:
      '200':
        description: Project unarchived successfully.
      '401':
        description: Admin identity could not be resolved from the request.
        content:
          application/json:
            schema: { $ref: '#/components/schemas/Error' }
      '404': { $ref: '#/components/responses/NotFound' }

# ===== Analytics (Admin) =====
/admin/analytics/skill-distribution:
  get:

```

```

tags: [Analytics]
summary: Get aggregate consultant counts per skill
description: Access Control -- Admin only.
responses:
  '200':
    description: Skill distribution.
    content:
      application/json:
        schema:
          type: array
          items:
            type: object
            properties:
              skillName: { type: string }
              consultantCount: { type: integer }

/admin/analytics/placement-ytd:
get:
  tags: [Analytics]
  summary: Get total placements made this year
  description: Access Control -- Admin only.
  responses:
    '200':
      description: Year-to-date placement count.
      content:
        application/json:
          schema:
            type: object
            properties:
              year: { type: integer }
              totalPlacements: { type: integer }

/admin/analytics/utilisation:
get:
  tags: [Analytics]
  summary: Get firm-wide consultant utilisation rate
  description: Access Control -- Admin only.
  responses:
    '200':
      description: Utilisation rate.
      content:
        application/json:
          schema:
            type: object
            properties:
              utilisationRate: { type: number, example: 0.72 }

/admin/analytics/utilisation/by-skill-category:
get:
  tags: [Analytics]
  summary: Get utilisation rates broken down by skill category
  description: Access Control -- Admin only.
  responses:
    '200':
      description: Utilisation by skill category.

```

```

    content:
      application/json:
        schema:
          type: array
          items:
            type: object
            properties:
              skillCategory: { type: string }
              utilisationRate: { type: number }

/admin/analytics/bench:
  get:
    tags: [Analytics]
    summary: Get count of consultants currently unplaced
    description: Access Control -- Admin only.
    responses:
      '200':
        description: Bench count.
        content:
          application/json:
            schema:
              type: object
              properties:
                benchCount: { type: integer }

/admin/analytics/bench/by-skill-category:
  get:
    tags: [Analytics]
    summary: Get bench counts broken down by skill category
    description: Access Control -- Admin only.
    responses:
      '200':
        description: Bench counts by skill category.
        content:
          application/json:
            schema:
              type: array
              items:
                type: object
                properties:
                  skillCategory: { type: string }
                  benchCount: { type: integer }

/admin/analytics/placements/by-skill-category:
  get:
    tags: [Analytics]
    summary: Get placement counts broken down by skill category
    description: Access Control -- Admin only.
    responses:
      '200':
        description: Placement counts by skill category.
        content:
          application/json:
            schema:
              type: array

```

```

        items:
          type: object
          properties:
            skillCategory: { type: string }
            placementCount: { type: integer }

/admin/analytics/cv-parsing-stats:
  get:
    tags: [Analytics]
    summary: Get aggregate CV parsing outcome statistics
    description: Access Control -- Admin only.
    responses:
      '200':
        description: CV parsing statistics.
        content:
          application/json:
            schema:
              type: object
              properties:
                totalParsed: { type: integer }
                successful: { type: integer }
                failed: { type: integer }

```